Remaining work and resumption handoff

E-Waste Compute Grid | Team + AI context | 9 October 2026

Resume at Step 5. Onboarding Steps 1-4 passed their LOCAL acceptance gates. Steps 5-7 remain open. This handoff changes documentation only.

Purpose: let a teammate understand the current system, obtain the tested source, implement the remaining work in order, and give the same context to an AI. The seven steps are the onboarding roadmap; they do not cover every production or model-quality goal.

Source checkpoint - read before pulling

Repository: https://github.com/dkush2024-dot/E-waste-computer-grid. Code checkpoint: ae8e64cb174d4fa41c63518265d375b18736b9a7. Onboarding Steps 1-4 and their acceptance evidence are committed. Pull main to receive this checkpoint and its companion handoff documentation.

The PDF explains the source; teammates obtain the code from Git. Compare the companion SHA-256 fingerprint after normalizing text line endings to LF. Environments, credentials, private configuration and .test-data are excluded. This publication does not deploy the project or message teammates.

Remaining milestone	Suggested lead	Completion evidence
5. Runtime distribution	Vaibhav + Nikhil	Pinned images, verified downloads, real preflight and failure tests
6. Installer and startup	Vaibhav + Chinmay	Clean-machine install/run, tray/startup, upgrade/uninstall tests
7. Physical acceptance	All; Nikhil records	Separate laptop, then concurrent multi-laptop work and recovery

Team roles

Kushagra: backend, API/database contracts and deployment configuration. Vaibhav: worker, Docker/runtime and packaging. Nikhil: datasets, model evaluation, benchmarks and evidence. Chinmay: desktop/dashboard experience. These are suggested ownership assignments from the existing team context, not messages sent to those people.

Reading guide

Architecture: p. 2. Evidence: p. 3. Source map: p. 4. Setup: p. 5. Step 5 tasks/tests: pp. 6-7. Installer: p. 8. Physical tests: pp. 9-10. Waste model: p. 11. Deployment/backlog: p. 12. Commands: p. 13. Copyable AI prompt: p. 14. Completion gates: p. 15.

What the project does today

Use this explanation when introducing the project to a teammate or judge

An administrator uploads a batch of images. The coordinator stores the inputs, creates a job, and splits it into independent tasks. Authorized laptop workers claim tasks, download their assigned images, run the approved CPU classifier in local Docker, and return JSON. The coordinator validates and combines the accepted results.

Component	Responsibility
Dashboard	Shows workers/jobs, uploads inputs, invites owners using pairing codes, downloads results.
Control plane + PostgreSQL	Stores the queue; authenticates nodes; assigns task leases; validates outputs; retries unfinished work.
Worker desktop + agent	Gets local owner approval, saves protected identity/policy, polls, downloads, runs and reports tasks.
Approved Docker workloads	MobileNetV2 ImageNet inference is real; dummy generates synthetic pipeline results.

Example: 1,000 images across four workers

At 10 images per task, the coordinator creates 100 tasks. Each worker claims one eligible task at a time and returns for another when finished. Tasks are claimed dynamically; each laptop is not guaranteed exactly 250 images. Faster or earlier workers may finish more tasks. Batch size changes task count, not the number of laptops.

Computation executes on each laptop using that laptop's own CPU and RAM. The coordinator transfers data and combines outputs. RAM is not pooled into one larger memory space, and an arbitrary application does not automatically become distributed. The workload must support independent batches.

Contracts to preserve

- One active task per worker; compatibility and required RAM affect admission. CPU quota is in logical CPU equivalents, not a percentage of the whole laptop.

- Pause stops new assignments while current work may finish. Stop cancels owned work; a fenced lease can be released for retry, with timeout recovery if delivery fails.

- Leases and idempotency prevent stale/duplicate accepted outputs. Execution can happen more than once after a failure; acceptance/accounting must happen once.

- Container CPU/RAM limits exclude Python, Docker and WSL overhead. Energy is an estimate from baseline power and accepted runtime, not measured savings.

Completed milestones and evidence

Current status takes precedence over older planning tables and exported PDFs

Step	Implemented and locally verified
1. Approval protocol	Expiring pairing, authenticated administrator invitation, owner approve/decline, stable identity, reconnect and revocation.
2. Desktop consent	Actual Windows invitation/decision, no work before approval, decline/expiry, Docker image jobs, scoped stop and same-identity app restart.
3. Resource policy	Owner CPU/RAM ceilings, Docker enforcement including zero extra swap, RAM admission, pause/drain, persisted pause, stop/retry and lease fencing.
4. Desktop protection	Responsive background operations, persistent controls, diagnostics, connection recovery, protected Windows state, settings restore and stop-before-disconnect.

Latest Step 4 checkpoint

- 61 worker tests passed. 33 backend tests passed with zero skips: 26 real PostgreSQL/HTTP integrations and seven unit checks. Backend build passed.

- Actual Windows UI approved 0.5 CPU and 0.75 GiB. Pause/resume, diagnostics, full app restart and disconnect were observed.

- Two genuine ten-photo jobs completed four Docker tasks and returned 20 predictions with exact filename coverage and zero failures, under one node and one approved pairing.

- Windows DPAPI roundtrip, plaintext migration, failed writes, tampering and same-account subprocess reconnect passed. Network-failure and several shutdown regressions used mocks; a real LAN partition was not exercised.

- Publication checks on 9 October: 61 worker tests, 33 backend tests with zero skips, 32 workload tests and frontend build/lint all passed again locally. CI now supplies a virtual display for Linux desktop tests and the Python path for real pairing integrations; hosted CI is a separate check.

Evidence a receiving AI should read

output/onboarding/step4-acceptance.md and step4-verification.json; step4-task-ownership.json; step4-desktop-job1-results.json and job2-results.json with their coverage checks. Read Step 3 acceptance/verification/Docker evidence for real active cancellation/retry and kernel ceilings. Keep historical records; write a new dated ledger for new work.

These are one-machine development results. They do not prove a clean installer, reboot, another Windows account, separate retired hardware, concurrent physical laptops, model accuracy, measured savings or AWS execution.

Step 4 test API on port 18082 and its desktop/agent were stopped after acceptance. The shared Docker/PostgreSQL runtime and ignored test fixture were retained. Do not assume any acceptance server is currently running.

Where to read and change the source

Relative paths below resolve inside the teammate's checkout

Path / module	Role and remaining-work relevance
worker/desktop.py	Connection/Diagnostics UI, consent controls, background work, settings and safe shutdown. Step 5 progress and Step 6 tray integration begin here.
worker/desktop_runtime.py	Preflight/agent subprocess, limits/environment, owned stop request and diagnostic tails. Packaging must launch the agent correctly when frozen.
worker/state_store.py; log_safety.py	DPAPI/atomic state and secret redaction. Preserve failures, per-grid identity and bounded logs.
worker/agent.py; api_client.py	Heartbeat, lease/cancel/report loop and authenticated API. Keep computation in the existing agent.
worker/task_executor.py; config.py; resource_limits.py	Allowlisted images, staging, isolation, container recovery and quota validation. Step 5 digest resolution must reach this path.
worker/pairing_client.py	Pairing/session recovery, approved credential save, pause and reconnect. Never distribute global admin/enrollment credentials.
control-plane/src/app.ts; pairing.ts; validation.ts; storage.ts	Assignment/auth/result contracts, pairing, exact input validation and local/S3 storage. Read before altering worker requests.
control-plane/prisma/schema.prisma	Node, PairingSession, Job, Task and Metric. No established production migration baseline.
frontend/src/App.tsx; src/services/api.ts	Dashboard invitation, batch upload, job polling and JSON download. Task-owner/attempt history views remain broader follow-up work.
workloads/ai-inference/; workloads/test-data/	Model/labels baked into inference image; pinned artifact/photo manifests; exact downloaded-result verifier.
infra/; scripts/; worker/tests/; control-plane/tests/	Compose, image Dockerfiles, launch/source handoff scripts and regression tests. Current source ZIP is CLI handoff, not desktop installer.

Read docs/WORKER_ONBOARDING.md and docs/api_contracts.md for current contracts. docs/MASTER_PROJECT_PLAN.md and PROJECT_REVIEW.md contain broader plans and historical findings; proposed modules/endpoints there are not automatically implemented. Older README/context statements about missing Git or pending protected storage may predate the latest ledger. Resolve contradictions against inspected code and dated evidence.

Get the same source and run locally

First receiving-team checkpoint; this page does not execute these commands

1. Obtain the tested changes

```
git clone https://github.com/dkush2024-dot/E-waste-computer-grid.git
cd E-waste-computer-grid
git status --short
git log -1 --oneline
```

For an existing clone, inspect local changes before pulling; preserve them, then run git pull --ff-only origin main. Confirm git merge-base --is-ancestor ae8e64c HEAD succeeds. This checkpoint contains Steps 1-4; the following documentation commit includes this handoff. Compare the LF-normalized source fingerprint before assuming an identical baseline.

2. Prepare a fresh worker environment and approved images

```
py -3.12 -m venv .venv-local
$workerPython = ".\.venv-local\Scripts\python.exe"
& $workerPython -m pip install -r worker/requirements.txt `
  -r workloads/ai-inference/requirements.txt
& $workerPython workloads/ai-inference/download_model.py
& $workerPython workloads/test-data/setup_sample_data.py
docker build -t ewaste/dummy-workload:v1 workloads/dummy-workload
docker build -t ewaste/ai-workload:v1 workloads/ai-inference
```

If the py launcher is unavailable, use the absolute path to an installed Python 3.12 executable for environment creation. Never copy an environment from another laptop. Docker must run Linux containers. Model/data setup validates cached hashes; preserve failures rather than manufacturing replacement data.

3. Start the coordinator on ONE computer

```
if (-not (Test-Path infra/.env)) {
  Copy-Item infra/.env.example infra/.env
}
docker compose --env-file infra/.env -f infra/docker-compose.yml up -d --build
Invoke-RestMethod http://localhost:8080/api/health
```

Configure unique private credentials in your local environment before sharing access. Open http://localhost:5173 and connect with that coordinator's admin token. Preserve existing configuration/data. Compose startup currently uses prisma db push; use a fresh development DB and review an existing-data upgrade instead of treating this as a production migration procedure.

Step 5: distribute verified workloads

Suggested lead: Vaibhav | Support: Nikhil; Chinmay for progress UI

Goal and current gap

Today workers need manually built images tagged ewaste/dummy-workload:v1 and ewaste/ai-workload:v1. The desktop verifies real execution but does not install/pull images. Deliver an explicit runtime preparation flow so another laptop can obtain the approved version and pass preflight without a source build.

Implementation sequence

- 5.1 Inventory supported Windows/CPU architectures and Docker/WSL prerequisites. Declare the initial target and test hardware; classify unsupported machines clearly. Check daemon reachability, Linux mode, free disk space and required images. Do not assume every retired laptop can run the runtime.

- 5.2 Build each approved workload from reviewed source with the pinned model/label manifest. Choose a team-owned registry, version the release and record the immutable image digest and platform. Registry publication needs the team's approved account and publishing scope.

- 5.3 Add a versioned workload manifest: workload ID, approved registry/image digest, platform, artifact checksums, runtime/minimum requirements and protocol/result version. Pin execution to the reviewed digest rather than trusting a movable v1 tag.

- 5.4 Add background image preparation with visible progress/status, timeouts, retry and cancellation. Use the trusted release manifest/allowlist, not an arbitrary image supplied by a job. Identify cached content by digest and reuse it.

- 5.5 Inspect the downloaded image/platform and run existing restricted preflight under the owner's selected ceilings. Mark readiness only after genuine inference succeeds. Bake model/labels into the image; computation remains network-disabled.

- 5.6 Carry the resolved digest consistently through desktop runtime, agent configuration and task executor. DesktopRuntime currently hardcodes v1 tags, so changing only Config.IMAGES is insufficient. Keep native mode explicitly development-only.

- 5.7 Document prerequisites, offline/cache behavior, failures, cleanup and update/rollback. Produce a dated sanitized Step 5 ledger with release digests and target device/runtime versions.

Expected deliverables

A reviewed release manifest; approved published workload images or an agreed offline import artifact; a desktop preparation/progress flow; allowlisted digest execution; meaningful failure tests; and a new setup runbook. These are proposed outputs, not files already present.

Downloading/preflight must not enroll or start job computation before owner approval. Runtime preparation must be explicit; do not silently substitute dummy results or weaken Docker execution after a failed check.

Step 5: failure cases and acceptance

Define success before changing the download/runtime path

Scenario	Required behavior / test
Docker absent, stopped or wrong mode	Explain the exact prerequisite and how to retry; no crash, false readiness or enrollment.
Unsupported architecture/runtime	Reject the incompatible release explicitly. Record tested support instead of guessing from CPU-core counts.
Image missing; registry denied/offline	Show preparation status and actionable error. Retry preserves identity/settings and consent.
Interrupted pull; cancellation; app close	UI remains responsive; only the app's operation stops. Retry succeeds; partially pulled/unverified content is not declared ready.
Disk full / insufficient free space	Fail clearly and leave state intact; bound cache/log use and document recovery.
Mutable tag replaced / digest mismatch	Refuse unapproved execution. Verify the selected digest/platform and artifact contract; demonstrate a wrong-version failure.
Cache hit, offline with valid cache	Skip redundant transfer and run real preflight. Document whether offline computation can contact its configured API.
Preflight failure under low RAM/CPU	Show the real diagnostic and limits. No silent dummy/native fallback; no advertising image capability without a successful probe.
Release update / rollback	Keep approved identity, frozen resource policy and pause. Document which release becomes active and when; avoid switching code mid-attempt.

Step 5 completion gate

- A fresh supported worker machine obtains the expected release with no manual Docker build from source.
- Manifest/digest/platform, input/model checks and actual restricted inference preflight pass; invalid images are rejected.
- The UI remains usable during download and cancellation; persisted secrets stay redacted and protected.
- After explicit approval, ten tracked photos complete with exact coverage and accepted metrics. Record real versus mocked test scope.
- The regression baseline still passes. Write output/onboarding/step5-verification.json plus a readable acceptance record, and update current docs.

Do not choose a completion percentage or promise a fixed duration from the step number. Step 5 depends on available registry access, supported hardware and the runtime environment. An offline team demo can use a documented, verified import route, but it must be labeled as such.

Step 6: package and install the desktop

Suggested leads: Vaibhav + Chinmay | Depends on stable Step 5 runtime flow

Goal and implementation tasks

- 6.1 Choose and document a Windows packaging/installer approach for the supported architecture. Bundle Python, Tkinter, required agent libraries and all local modules; exclude source-only assumptions, private state and coordinator services.

- 6.2 Adapt the frozen entry points. Today DesktopRuntime uses sys.executable plus worker/agent.py; in a packaged executable sys.executable may refer to the GUI itself. Provide a defined agent/preflight mode or separate bundled agent, with robust paths and controlled subprocess lifecycle.

- 6.3 Build an installer with version metadata, shortcuts, sensible install/state directories and prerequisite diagnostics. Normal owner interaction should not require a terminal or project checkout. Docker/WSL remains an external prerequisite unless an explicit supported provisioning flow is implemented.

- 6.4 Add tray state and Start/Stop/Pause/Resume controls. Define window close versus tray close versus Exit clearly so a user knows whether computation continues. Keep task computation in the agent, not the tray/UI thread.

- 6.5 Make login startup opt-in. Define whether startup opens the window or starts an already approved agent. Never bypass first consent, revive a revoked identity, or override saved pause. Show the enabled setting and provide a clear disable path.

- 6.6 Ensure desktop and agent instance locks work in the installed app. Upgrades/restarts preserve the per-grid DPAPI identity, owner limits, name and pause without creating extra nodes. Do not copy credentials between accounts.

- 6.7 Implement scoped upgrade/uninstall. Stop only this app's agent/containers/startup entries. Explain whether owner state is retained or explicitly deleted; do not remove shared Docker, unrelated containers or coordinator data.

- 6.8 Produce release artifacts, checksums and installation/recovery instructions. Code signing, trusted updates and long-term support are separate release hardening tasks; do not advertise them before they exist.

Acceptance matrix

Test	Success condition
Clean Windows machine	Installs and runs with no system Python/project source; missing Docker gives a clear message.
Install path with spaces; fresh account	UI and packaged agent/preflight start from correct paths; state belongs to the current account.
Consent, pause, reboot, update	Approval is required initially; stable identity and resource policy persist; startup respects saved pause.
Double launch; uninstall during work	No duplicate agent; only owned work is stopped; unrelated processes/containers survive.

Step 7: prove physical distribution

All teammates | Nikhil records the evidence | Begin with one external worker

A. Prepare a real fleet

Use one coordinator laptop and at least three distinct physical worker laptops for the main evidence run. With four laptops total, one can coordinate and three can compute. With five, one can coordinate and four can compute. The coordinator may also run an explicitly approved worker, but record that role and avoid counting it as another device.

- Put devices on a mutually reachable LAN; a common Wi-Fi name does not guarantee communication because client isolation can block it. Record the current coordinator IPv4 and verify health from each worker.

- Choose distinct names (Laptop B/C/D/E), independent state directories and approved identities. Record hardware/OS/runtime/app/image version, consent, connection time and whether the device is genuinely retired.

- Enable only the intended coordinator API access. Workers contact it outbound; they do not need incoming worker ports or automated LAN scanning. Keep PostgreSQL local/private.

- Verify each worker's supported Docker runtime, real preflight, CPU/RAM ceiling, writable state and signed input/output reachability. The dataset stays uploaded at the coordinator; do not process photos manually on each laptop.

B. Submit a measurable job

Start with ten genuine images and the existing coverage verifier. Then use enough unique tracked inputs/tasks for several workers to overlap: for example 1,000 photos at chunk size 10 gives 100 tasks. Choose batches/limits by measured behavior, not a guaranteed speedup. Start all approved workers before submitting the larger run.

Normal image jobs require at least the default 2 GiB task admission requirement. A worker approved below that remains ineligible unless the job explicitly requests a compatible, tested lower requirement. Do not increase owner ceilings on the server merely to make a queued job run.

C. Prove the distribution, not just COMPLETED

- Record distinct physical machines and node IDs. Show that each required worker accepted at least one task and that execution overlaps in time.

- Capture sanitized task-owner/attempt/timestamp records. Current dashboard shows fleet/job summaries, not a full accepted-attempt timeline; use reviewed read-only evidence queries, or implement a bounded secret-free read view as a separately scoped improvement.

- Verify every submitted image appears exactly once as prediction or explicit error; check terminal job/task counters and one accepted metric per task. Unexpected duplicates/missing identities are failures.

- Record elapsed job time separately from summed worker execution time. Compare identical useful work on one worker and the fleet before claiming speedup.

Step 7: failure and recovery matrix

Repeat on real physical devices; preserve exact input/result accounting

Test	Expected outcome / evidence
Decline or expire invitation	No computing/worker identity from that decision; expiry can request a fresh session.
Normal app restart and physical reboot	Same approved node, limits and pause. Explicit Start/Resume rules respected; no copied credentials.
Different Windows account or copied state	Protected state is not silently reset/re-enrolled. Fail with recovery guidance; require that account/device's own approval.
Pause during active work	Only the current task drains; no new assignment. Resume completes the remaining queue.
Stop during download/execution/upload	Owned activity cancels safely; partial attempts do not count as successful outputs. Lease release/expiry allows bounded retry.
Abrupt worker/process/power loss	An unfinished lease expires and another eligible worker retries it. Verify one accepted result/metric and correct final counts.
Late response from old attempt	Old lease cannot overwrite the new accepted result or inflate metrics. Record the recovered attempt.
LAN loss and reconnect	UI remains responsive, logs redact secrets, identity persists; verify recovery and expiry semantics after restoring reachability.
API restart while work exists	Durable queue survives; workers reconnect and task ownership/accounting remains correct. Record lease/signing URL expiry behavior.
Revocation during active work	New claims and completion are denied; agent stops when it observes denial. No automatic re-enrollment.
Corrupt input, container OOM/timeout	Bounded explicit errors/retries; job finishes with correct success/partial/failure accounting, not fabricated predictions.
Unrelated container / duplicate launch	Only owned containers stop; another app/container survives. Same state directory cannot launch a duplicate worker.

Timing and acceptance

Use actual configured lease, sweep, offline and retry settings. Default API lease is 300 seconds, sweep 10 seconds and offline threshold 90 seconds; offline status alone is not reassignment. An isolated demo may use reviewed shorter timings, but do not mix those results with default behavior or reset real data.

Capture failure time, detection/retry/completion time, old/new owner/lease evidence, versions and exact coverage. Repeated owner stops consume the normal retry budget and can eventually fail a task. Keep raw evidence with secrets removed. Freeze the release only after a repeatable multi-device rehearsal passes.

Additional goal: waste classification

Separate model/data milestone; this is not supplied by onboarding completion

Current inference uses MobileNetV2 with 1,000 ImageNet classes. It does not yet reliably label wet waste, electronic waste, plastic or general non-biodegradable waste. A successful distributed job proves orchestration and accounting, not classification accuracy.

Tasks required for the requested waste use case

- W1. Define a stable labeling policy. Plastic is usually non-biodegradable and electronics can contain plastic, so these categories overlap. Choose a documented hierarchy or mutually exclusive primary labels; define contaminated/mixed/unknown items and when human review is required.

- W2. Obtain licensed, representative labeled images for each category; record provenance and consent/privacy requirements. Split train/validation/test without duplicates or near-duplicate leakage. Keep a held-out evaluation set with ground truth.

- W3. Train/fine-tune or select a suitable CPU-compatible model; evaluate per-class precision/recall, confusion matrix and low-confidence/mixed-waste behavior. Set an agreed accuracy target before evaluating; the target is not established yet.

- W4. Export/version the inference artifact with labels, checksums and preprocessing metadata. Confirm compatibility with the approved ONNX runtime and old target CPUs. Build a new reviewed Docker image; do not simply rename ImageNet labels.

- W5. Preserve per-image identity, class ID/name, confidence and explicit error accounting. Update workload/version/capability contracts, validators and tests if necessary. Prevent workers with incompatible model releases from mixing outputs in one job.

- W6. Add useful output: CSV/JSON with filename, predicted waste class, confidence and review flag; category totals and evaluation report. Keep a mapping from coordinator-assigned names to original files. Folder sorting, if added, must handle duplicates and uncertain predictions explicitly.

- W7. Demonstrate the same waste batch on multiple devices and validate both exact coverage and label quality. Benchmark throughput separately from model accuracy. Document the model's limits and review workflow.

How the output becomes useful

For a waste audit, predictions can create a searchable inventory of photographed items, category counts, and a review queue. Reviewed labels could support an operator's sorting/routing report or a downstream application. The current system returns metadata; it does not physically sort waste, control a robot or prove a disposal decision.

If the hackathon demo promises these waste categories, this model/data milestone is required in addition to Steps 5-7. If the demo is general distributed image classification, describe the current model accurately and leave this extension clearly labeled.

Additional deployment and release backlog

Keep these gates separate from the three remaining onboarding milestones

Area	Remaining work / evidence
AWS demonstration, if included	Select a reviewed architecture/account/region, configure unique secrets and HTTPS, deploy the coordinator, and verify real worker-to-API and storage transfers. Existing S3 code is not live cloud evidence. Record costs, inventory and teardown.
DB release operations	Establish versioned Prisma migration/baseline, back up existing data and rehearse restore/upgrade. Current db push/additive SQL is a prototype path, not production rollback evidence.
Dashboard visibility	Add bounded task-owner, attempt/retry, timing, detailed results and queued-reason views if needed for the demo. Preserve secret-free responses; the current sync endpoint is not a full history API.
Deployment security	Review TLS, private DB/storage, unique credentials, auth/rotation, quotas and proxy/shared rate limiting. Restrict legacy enrollment for approval-only use. Public volunteer admission needs a separate threat model/result-trust design.
Operational reliability	Readiness and queue/sweep/error signals, API/DB/storage outage rehearsals, bounded logs/retention and an operator recovery runbook. Verify real deployment behavior rather than adding decorative metrics.
Performance and energy	Measure identical work with one versus multiple devices, including transfer/startup/retries. Baseline-power estimates are not energy-saving or carbon evidence. Record actual retired-device reuse separately.
CI and release reproducibility	Run CI against the final source, clean-clone setup, fresh-device support matrix, release tag/checksums and dependency inventory. Existing CI definition alone is not an observed successful remote release run.
Optional advanced work	Live-load/energy scheduling, adaptive chunks/data locality, more workloads or a broker only after profiling and a scoped requirement. Standalone scoring/benchmark helpers are not integrated scheduling.

AWS boundary

No AWS deployment was created or verified during Step 4 or this handoff. Treat credentials, publication and deployment permissions as separate user-approved scope. Recheck the current hackathon's actual eligibility requirements using the official event page when choosing a route; this document does not certify eligibility.

Do the LAN/physical acceptance before claiming a cloud fleet. When cloud is included, repeat the same real-image/recovery checks using the actual HTTPS API and chosen storage. Record the deployed commit/image version and the exact resource/credential boundary, without placing secrets in the PDF.

Commands and contracts for resuming

PowerShell examples; replace machine-specific values before running

Connect another laptop to the coordinator

For trusted LAN development, set API_BIND_ADDRESS=0.0.0.0 in the coordinator's private infra/.env and recreate the control-plane service. Keep DB bound to loopback. PUBLIC_API_URL must produce input/output URLs reachable by workers; relative /api URLs are supported. Open the dashboard locally on the coordinator unless you intentionally configure its LAN access.

```
# On each WORKER laptop; use the actual coordinator IPv4/port.
$gridApi = "http://SERVER_IP:8080/api"
Invoke-RestMethod "$gridApi/health"
$workerPython = ".\.venv-local\Scripts\python.exe"
& $workerPython worker/desktop.py --api-url $gridApi --allow-http
```

SERVER_IP is a placeholder, not a hostname discovered by the app. Each owner prepares, shares only the code, and approves after the admin invites it. For public/HTTPS use the trusted HTTPS address and omit --allow-http. No deployment is needed for a reachable LAN demo. Do not use the legacy Start-Worker enrollment-token flow as a substitute for owner approval.

Run the relevant regression baseline

```
& $workerPython -m unittest discover -s worker/tests -v
# Choose a PRIVATE, DEDICATED development/test PostgreSQL database.
$env:TEST_DATABASE_URL = "YOUR_TEST_DATABASE_CONNECTION_STRING"
$env:TEST_PYTHON = (Resolve-Path $workerPython).Path
Push-Location control-plane
npm ci
npm test
Pop-Location
Push-Location frontend
npm ci
npm run build
npm run lint
Pop-Location
& $workerPython -m unittest discover -s workloads/tests -v
```

Stop relevant dev processes before npm ci if Windows native dependencies are locked. Without TEST_DATABASE_URL, backend integrations skip; report skips explicitly. Test schemas are isolated, but use only a test DB. Do not copy development credentials into these examples or reset an existing database to make tests pass.

Verify downloaded real-photo output

```
& $workerPython workloads/test-data/verify_job_results.py `
  "C:/path/to/downloaded-job.json" --report "coverage-check.json"
```

That CLI verifies the existing ten-photo manifest with chunk size five. For a different dataset/chunk size, extend the verifier deliberately or call its verify_job function with the exact submitted names/chunk size. Do not apply the ten-photo expected set to a 1,000-image job. Output validation is not visual label evaluation.

Copyable prompt for the next AI

Share the current source snapshot and companion Markdown along with this PDF

The following is a suggested USER prompt for a teammate to copy. It is not authorization for this document-generation task to implement or deploy anything.

Paste this prompt into the AI with access to the actual repository, after confirming that it contains the tested onboarding changes.

```
You are helping our team resume E-Waste Compute Grid. Read this handoff,
the current repository and applicable AGENTS/skill instructions first.
Repository: https://github.com/dkush2024-dot/E-waste-computer-grid.
The code checkpoint is ae8e64cb174d4fa41c63518265d375b18736b9a7.
Pull main and verify that this commit is an ancestor of my HEAD.
Compare the supplied source fingerprint after normalizing text to LF.
If files differ or are missing, report that before assuming this baseline.

Read README.md, docs/WORKER_ONBOARDING.md, docs/api_contracts.md,
docs/MASTER_PROJECT_PLAN.md, and output/onboarding/step4-acceptance.md
with its verification JSON. Current source and dated evidence supersede
older planning tables/PDFs. Inspect code before claiming a feature exists.

Steps 1-4 passed LOCAL acceptance: pairing/owner consent, desktop,
resource policy/pause/stop, Windows DPAPI state and diagnostics.
Latest evidence: 61 worker tests; 33 backend tests with zero skips;
two ten-photo Docker jobs, four accepted tasks, exact coverage,
same node after full app restart. This is one-machine evidence.

Continue STEP 5 ONLY: verified workload-image distribution, trusted
digest/version manifest, Docker/WSL diagnostics, responsive download
progress/retry/cancel/cache, and real preflight. Inspect DesktopRuntime's
hardcoded image tags and Config.IMAGES/TaskExecutor before changing them.
Identify the registry/runtime decisions needing team input. Complete
independent local implementation/testing within this task's scope;
publication/deployment requires the team's authorized account and scope.

Preserve owner approval, per-grid DPAPI identity, frozen CPU/RAM limits,
saved pause, scoped cleanup, leases, bounded retries and exact input
accounting. Do not ship global tokens, reset data, copy credentials,
silently fall back to dummy/native, or rewrite the scheduler unnecessarily.
Keep the agent as the computation engine. Do not start Steps 6-7 or
waste-model training merely because they appear in this handoff.

Run meaningful affected tests and actual runtime checks; enable the
test DB for integration and report skips. Record commands, versions,
release digests, changed files, actual versus mocked evidence and limits.
Update current docs and create a dated Step 5 acceptance/verification
ledger. Do not call a source ZIP an installer or a local run a physical
fleet/AWS/model-accuracy/energy-saving result.
```

Completion checklist and team handover

Use these gates to decide what can be claimed at the next checkpoint

Gate	Required deliverable / evidence
Source handoff	Receiving checkout has the reviewed Steps 1-4 changes. Record commit or snapshot/fingerprint; no copied private state.
Step 5	Approved digests/manifest, preparation UI, cancellation/retry/cache failures, real preflight and genuine-photo results.
Step 6	Versioned installer, clean-machine proof without Python/source, tray/startup policy, reboot/upgrade/uninstall and scoped process cleanup.
Step 7	Separate retired laptop plus at least three physical contributing workers, overlapping tasks, failure recovery and exact result accounting.
Waste goal, when promised	Defined categories, licensed dataset splits, held-out evaluation, versioned waste model and useful reviewed result exports.
AWS goal, when promised	Actual deployed version, HTTPS API/storage transactions from physical workers, recovery results and resource/cost/teardown record.
Release claims	Repeatable demo, bounded known limitations, accurate measured/estimated labels, final CI/setup and release identity.

Record after each milestone

- Problem and resulting behavior; owners and files changed; API/schema/model/installer impact.
- Exact commit/snapshot, dependency/runtime/OS/image versions, commands and isolated test environment.
- Passed/failed/skipped checks; physical device identities; accepted task/attempt/output accounting; redacted logs.
- Limits and unresolved failures. Point to actual evidence rather than marking every checklist item done.
- Update docs/WORKER_ONBOARDING.md and current team context. Preserve older ledgers and regenerate a new dated handoff; do not overwrite history with unsupported claims.

Files to send

Send this PDF for reading; its Markdown companion for AI copy/paste; the source fingerprint for comparison; and access to the reviewed current source/evidence. Do not send .venv, .env, credentials.json, pairing.json, DPAPI state or .test-data. The existing worker-only source ZIP does not contain the full coordinator/dashboard/desktop handoff.

How to explain current readiness

Core computing and local owner controls work. Distribution, installation and physical proof remain. Waste-model quality and cloud/production gates are separate. Four completed onboarding steps is not a reliable percentage of total effort. No application code or live service changed for this handoff.